

ClinicServer — API do zarządzania przychodnią weterynaryjną

REST API dla przychodni weterynaryjnej zbudowane na ASP.NET Core 10.0 i Entity Framework Core, z autoryzacją JWT, szyfrowaniem AES-256-GCM, haszowaniem haseł Argon2 i telemetrią Azure Monitor.

Spis treści

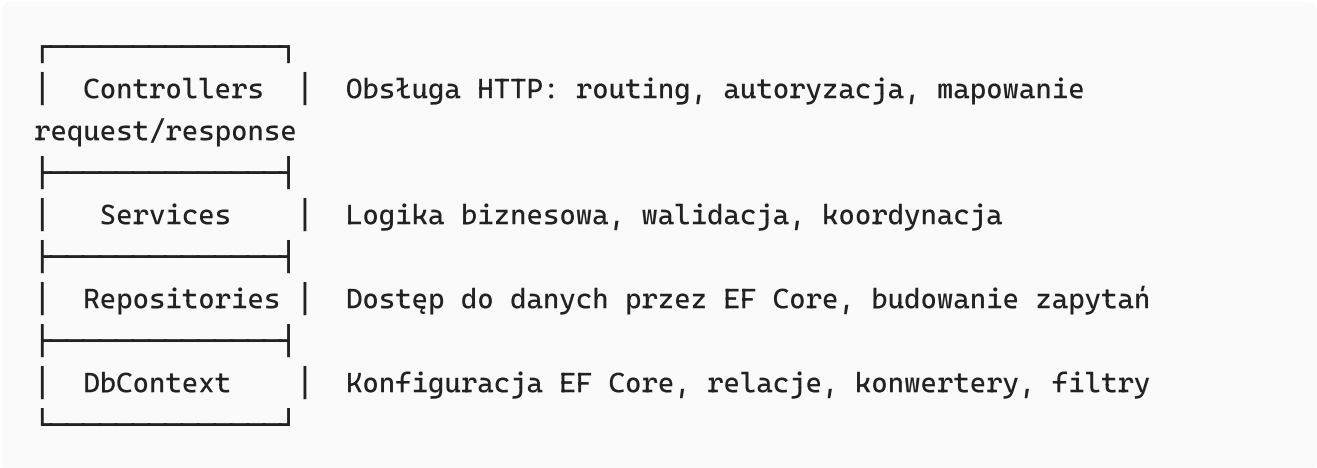
- [Stos technologiczny](#)
- [Architektura](#)
- [Struktura projektu](#)
- [Schemat bazy danych](#)
- [Endpointy API](#)
 - [Health](#)
 - [Autoryzacja](#)
 - [Gatunki i rasy](#)
 - [Klienci](#)
 - [Zwierzęta](#)
 - [Pracownicy](#)
 - [Leki](#)
 - [Magazyn](#)
 - [Wizyty](#)
 - [Karta medyczna](#)
 - [Diagnostyka](#)
 - [Słowniki](#)
- [Bezpieczeństwo](#)
- [Konfiguracja](#)
- [Uruchomienie](#)

Stos technologiczny

Kategoria	Technologia	Zastosowanie
Framework	.NET 10.0 (ASP.NET Core)	Serwer HTTP, pipeline middleware, DI
Język	C# 13	Logika aplikacji
ORM	Entity Framework Core 10.0	Mapowanie obiektowo-relacyjne, migracje
Baza danych	PostgreSQL (Npgsql)	Główna baza danych
Auth	JWT Bearer (2 schematy) + Refresh Token	Autoryzacja bezstanowa z rotacją tokenów
Haszowanie haseł	Argon2	Haszowanie haseł
Szyfrowanie	AES-256-GCM	Szyfrowanie pól wrażliwych w bazie danych
Walidacja	FluentValidation 12	Walidacja DTO z wstrzykiwaniem zależności
Telemetria	Azure Monitor + OpenTelemetry	Śledzenie rozproszone, metryki, logowanie
Storage plików	Local (dev) / Azure Blob (prod)	Przechowywanie plików diagnostycznych
Rate Limiting	ASP.NET Core	Zabezpieczenie przed wysyłaniem zbyt dużej ilości zapytań

Architektura

Aplikacja wykorzystuje architekturę **Controller** → **Service** → **Repository**:

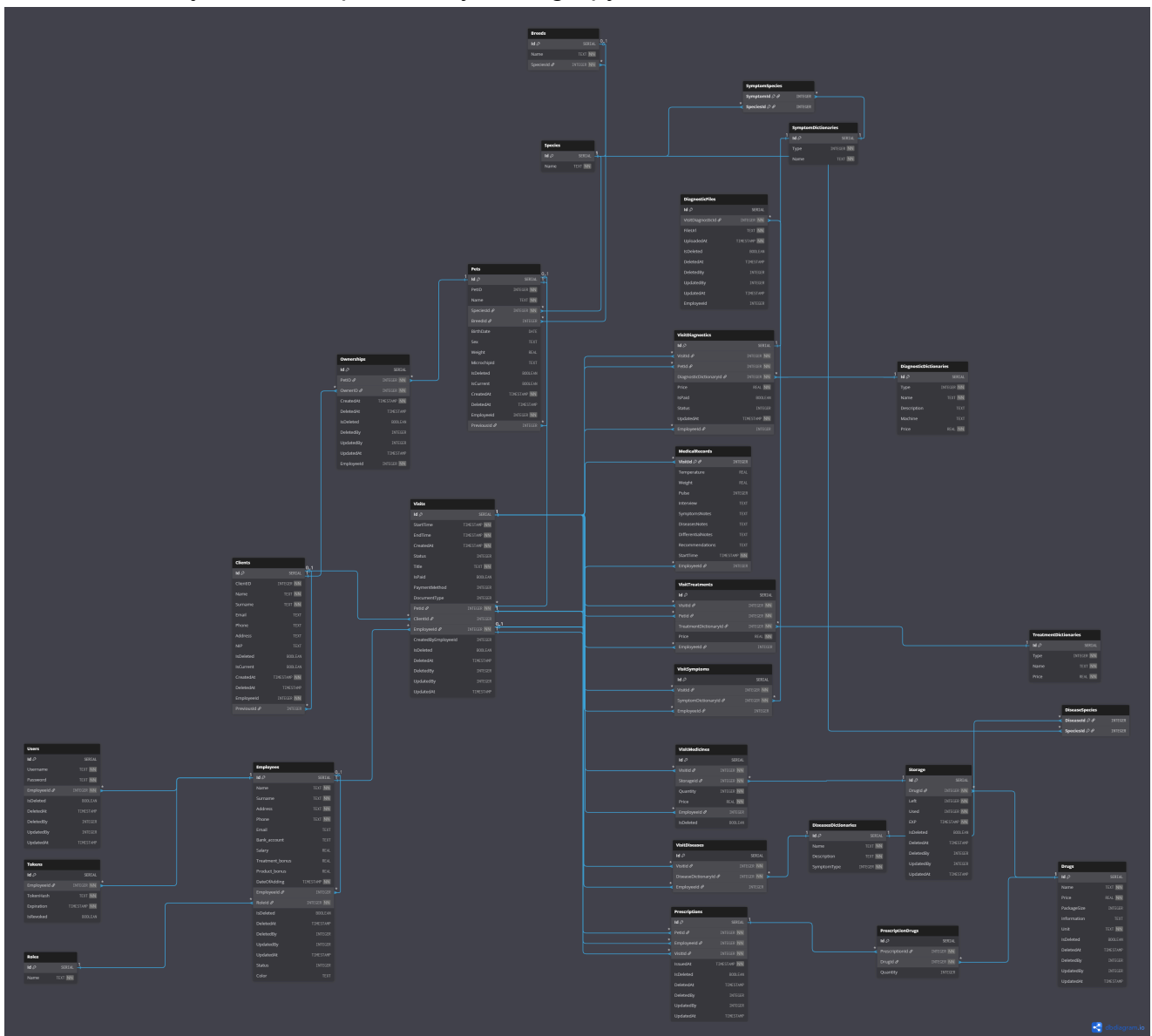


Struktura projektu

```
ConsoleApp4/
├─ Program.cs                # Punkt wejścia, pipeline HTTP
├─ ClinicServer.csproj       # Plik projektu
├─ appsettings.json         # Konfiguracja ogólna
├─
├─ Controllers/             # Kontrolery API
│   ├── AnimalController.cs  # Gatunki i rasy
│   ├── AuthController.cs    # Rejestracja, logowanie, refresh,
logout
│   ├── ClientController.cs  # Klienci + relacjami ze zwierzętami
│   ├── DiagnosticsController.cs # Badania diagnostyczne + pliki
│   ├── DictionaryController.cs # CRUD słowników
│   ├── DrugController.cs    # CRUD leków
│   ├── EmployeeController.cs # CRUD pracowników + status
│   ├── MedicalRecordsController.cs # Karty medyczne wizyt
│   ├── PetController.cs     # CRUD zwierząt + relacje + historia
│   ├── StorageController.cs # Zarządzanie magazynem
│   ├── UtilsController.cs   # Health check
│   ├── VisitController.cs   # CRUD wizyt + status + płatności
│   └─ LogicResult.cs        # Model standaryzowanej odpowiedzi
API
├─
├─ Models/                  # Encje EF Core
├─
├─ DTOs/                    # Obiekty request/response
├─ Data/                    # Konfiguracja EF Core
│   ├── AppDbContext.cs     # DbSets, relacje, indeksy, filtry
│   └─ EncryptionConfigurator.cs # Konfiguracja szyfrowania
├─
├─ Repositories/            # Warstwa dostępu do danych
├─ Services/                 # Warstwa logiki biznesowej
├─ Extensions/              # Konfiguracja pipeline i DI
├─ Utils/                   # Narzędzia przekrojowe
├─ Migrations/              # Migracje EF Core
└─ Properties/launchSettings.json # Konfiguracja uruchomieniowa
```

Schemat bazy danych

Baza składa się z **27 tabel** podzielonych na grupy domenowe.



Domeny główne

Tabela	Opis	Kluczowe relacje
Species	Katalog gatunków zwierząt (pies, kot, królik, itd.)	→ Breeds, SymptomSpecies, DiseaseSpecies
Breeds	Rasy przypisane do gatunków	→ Species
Pets	Zwierzęta pacjenci	→ Species, Breed, Employee
Clients	Właściciele zwierząt — dane kontaktowe, NIP	→ Employee
Employees	Pracownicy kliniki — dane osobowe, pensja, bonusy, status obecności	→ Role, WhoAdded
Roles	Role użytkowników (Admin, Recepcja, Technik, Lekarz)	→ Employees
Users	Konta logowania — username + hash hasła (Argon2)	→ Employee
Tokens	Przechowywanie refresh tokenów (hash HMAC-SHA256)	→ Employee

Kliniczne

Tabela	Opis	Kluczowe relacje
Visits	Wizyty lekarskie — terminy, status, płatności, dokumenty	→ Pet, Client, Employee
MedicalRecords	Karta medyczna wizyty — parametry życiowe, wywiad, notatki, zalecenia	→ Visit (1:1)
VisitDiagnostics	Badania diagnostyczne zlecone podczas wizyty	→ Visit, Pet, DiagnosticDictionary
VisitTreatments	Zabiegi wykonane podczas wizyty	→ Visit, Pet, TreatmentDictionary
VisitSymptoms	Symptomy zgłoszone podczas wizyty	→ Visit, SymptomDictionary
VisitMedicines	Leki przepisane podczas wizyty (z magazynu)	→ Visit, Storage
VisitDiseases	Choroby zdiagnozowane podczas wizyty	→ Visit, DiseaseDictionary

Słowniki (dane referencyjne)

Tabela	Opis
DiagnosticDictionaries	Dostępne badania diagnostyczne (morfologia, RTG, MRI, itd.) z cenami
SymptomDictionaries	Katalog symptomów kategoryzowanych według układów (oddechowy, pokarmowy, itp.)
TreatmentDictionaries	Dostępne zabiegi (szczepienie, operacja, konsultacja, itd.) z cenami
DiseasesDictionaries	Katalog chorób powiązany z typami symptomów
Drugs	Katalog leków z jednostką i rozmiarem opakowania

Magazyn i recepty

Tabela	Opis
Storage	Partie leków w magazynie — ilość, użycie, data ważności
Prescriptions	Recepty wystawione podczas wizyt
PrescriptionDrugs	Relacja M:N między receptą a lekiem z ilością

Tabele pośrednie

Tabela	Opis
Ownerships	Przypisania zwierząt do właścicieli (M:N)
SymptomSpecies	Które symptomy dotyczą których gatunków
DiseaseSpecies	Które choroby dotyczą których gatunków
DiagnosticFiles	Pliki wynikowe diagnostyki

Endpointy API

Wszystkie endpointy są poprzedzone ścieżką `/api` .

Health

Metoda	Ścieżka	Auth	Opis
GET	/api/	—	Sprawdzenie czy serwer odpowiada

Autoryzacja

Metoda	Ścieżka	Opis
POST	/api/Auth/register	Rejestracja nowego użytkownika
POST	/api/Auth/login	Logowanie, zwraca JWT + refresh token (ciasteczko HttpOnly)
POST	/api/Auth/refresh	Rotacja refresh tokena, nowy JWT
POST	/api/Auth/logout	Unieważnienie refresh tokena, usunięcie ciasteczka

Gatunki i rasy

Metoda	Ścieżka	Opis
GET	/api/Animals/species	Lista wszystkich gatunków
POST	/api/Animals/species	Dodanie nowego gatunku
PATCH	/api/Animals/species/{id}	Aktualizacja gatunku
DELETE	/api/Animals/species/{id}	Usunięcie gatunku
GET	/api/Animals/species/{speciesId}/breeds	Lista ras dla danego gatunku
POST	/api/Animals/species/{speciesId}/breeds	Dodanie nowej rasy
PATCH	/api/Animals/breeds/{breedId}	Aktualizacja rasy
DELETE	/api/Animals/breeds/{breedId}	Usunięcie rasy

Klienci

Metoda	Ścieżka	Opis
POST	/api/Client	Dodanie nowego klienta
GET	/api/Client	Lista klientów (paginacja, filtrowanie)

Metoda	Ścieżka	Opis
GET	/api/Client/{id}	Szczegóły klienta po ClientID
PUT	/api/Client/{id}	Aktualizacja danych klienta
DELETE	/api/Client/{id}	Miękkie usunięcie klienta
GET	/api/Client/specificRecord/{recordId}	Pobranie klienta po ID rekordu
POST	/api/Client/{ownerId}/pets/{petId}	Przypisanie zwierzęcia do właściciela
DELETE	/api/Client/{ownerId}/pets/{petId}	Usunięcie przypisania zwierzęcia
GET	/api/Client/{ownerId}/pets	Lista zwierząt należących do klienta

Zwierzęta

Metoda	Ścieżka	Opis
POST	/api/Pet	Dodanie nowego zwierzęcia
GET	/api/Pet	Lista zwierząt (paginacja, filtrowanie)
GET	/api/Pet/{id}	Szczegóły zwierzęcia po PetID
PUT	/api/Pet/{id}	Aktualizacja danych zwierzęcia
DELETE	/api/Pet/{id}	Miękkie usunięcie zwierzęcia
GET	/api/Pet/specificRecord/{id}	Pobranie zwierzęcia po ID rekordu
POST	/api/Pet/{petId}/owners/{ownerId}	Przypisanie właściciela do zwierzęcia
DELETE	/api/Pet/{petId}/owners/{ownerId}	Usunięcie właściciela ze zwierzęcia
GET	/api/Pet/{petId}/owners	Lista właścicieli zwierzęcia
GET	/api/Pet/{id}/records	Historia wizyt zwierzęcia
GET	/api/Pet/{petId}/prescriptions	Lista recept dla zwierzęcia

Pracownicy

Metoda	Ścieżka	Opis
POST	/api/Employee	Dodanie nowego pracownika
GET	/api/Employee	Lista pracowników (paginacja, filtrowanie)
GET	/api/Employee/{id}	Szczegóły pracownika
PUT	/api/Employee/{id}	Aktualizacja danych pracownika
DELETE	/api/Employee/{id}	Miękkie usunięcie pracownika
PATCH	/api/Employee/status	Zmiana statusu obecności pracownika

Leki

Metoda	Ścieżka	Opis
POST	/api/Drug	Dodanie nowego leku do słownika
GET	/api/Drug	Lista leków (paginacja, filtrowanie)
GET	/api/Drug/{DrugId}	Szczegóły leku
PUT	/api/Drug/{DrugId}	Aktualizacja danych leku
DELETE	/api/Drug/{DrugId}	Miękkie usunięcie leku
GET	/api/Drug/specificRecord/{RecordId}	Pobranie leku po ID rekordu

Magazyn

Metoda	Ścieżka	Opis
POST	/api/Storage	Dodanie nowej partii leku do magazynu
GET	/api/Storage	Lista pozycji magazynowych (paginacja, filtrowanie)
GET	/api/Storage/{StorageId}	Szczegóły pozycji magazynowej
PATCH	/api/Storage/{StorageId}	Aktualizacja pozycji magazynowej
DELETE	/api/Storage/{StorageId}	Miękkie usunięcie pozycji magazynowej
GET	/api/Storage/specificRecord/{RecordId}	Pobranie pozycji magazynowej po ID rekordu

Wizyty

Metoda	Ścieżka	Opis
POST	/api/Visit	Umówienie nowej wizyty
GET	/api/Visit	Lista wizyt (filtrowanie po dacie, statusie, zwierzęciu, lekarzu)
GET	/api/Visit/{id}	Szczegóły wizyty
PUT	/api/Visit/{id}	Aktualizacja danych wizyty
DELETE	/api/Visit/{id}	Miękkie usunięcie wizyty
PATCH	/api/Visit/{id}/status	Zmiana statusu wizyty
PATCH	/api/Visit/{id}/paid	Oznaczenie wizyty jako opłaconej

Karta medyczna

Metoda	Ścieżka	Opis
GET	/api/MedicalRecords/{visitId}	Pobranie karty medycznej dla wizyty
POST	/api/MedicalRecords/{visitId}	Utworzenie karty medycznej
PATCH	/api/MedicalRecords/{visitId}	Częściowa aktualizacja karty medycznej
POST	/api/MedicalRecords/{visitId}/treatments	Dodanie zabiegu do wizyty
GET	/api/MedicalRecords/{visitId}/treatments	Lista zabiegów dla wizyty
DELETE	/api/MedicalRecords/treatments/{treatmentId}	Usunięcie zabiegu
POST	/api/MedicalRecords/{visitId}/diseases	Dodanie chorób do wizyty
GET	/api/MedicalRecords/{visitId}/diseases	Lista chorób dla wizyty
DELETE	/api/MedicalRecords/desises/{desiseId}	Usunięcie choroby
POST	/api/MedicalRecords/{visitId}/symptoms	Dodanie symptomów do wizyty
GET	/api/MedicalRecords/{visitId}/symptoms	Lista symptomów dla wizyty
DELETE	/api/MedicalRecords/symptoms/{symptomId}	Usunięcie symptomu
POST	/api/MedicalRecords/{visitId}/medicines	Dodanie leku do wizyty
GET	/api/MedicalRecords/{visitId}/medicines	Lista leków dla wizyty
DELETE	/api/MedicalRecords/medicines/{medicineId}	Usunięcie leku
POST	/api/MedicalRecords/{visitId}/prescription	Utworzenie recepty z lekami
GET	/api/MedicalRecords/{visitId}/prescription	Pobranie recepty dla wizyty
DELETE	/api/MedicalRecords/prescription/{prescriptionId}	Usunięcie recepty
GET	/api/MedicalRecords/{visitId}/cost	Obliczenie całkowitego kosztu wizyty

Metoda	Ścieżka	Opis
PATCH	/api/MedicalRecords/{visitId}/finish	Zakończenie wizyty

Diagnostyka

Metoda	Ścieżka	Opis
POST	/api/Diagnostics	Zlecenie badania diagnostycznego
GET	/api/Diagnostics	Lista badań (paginacja, filtrowanie)
PATCH	/api/Diagnostics/{id}	Aktualizacja statusu badania
DELETE	/api/Diagnostics/{id}	Usunięcie badania wraz z plikami
POST	/api/Diagnostics/{id}/files	Załączenie plików do badania
GET	/api/Diagnostics/{id}/files	Pobranie URL-i do plików
DELETE	/api/Diagnostics/files/{fileId}	Usunięcie pojedynczego pliku

Słowniki

Metoda	Ścieżka	Opis
POST	/api/Dictionary/diagnostic	Dodanie typu badania diagnostycznego
GET	/api/Dictionary/diagnostic	Lista wszystkich typów badań
PUT	/api/Dictionary/diagnostic/{id}	Aktualizacja typu badania
DELETE	/api/Dictionary/diagnostic/{id}	Usunięcie typu badania
POST	/api/Dictionary/symptom	Dodanie symptomu do słownika
GET	/api/Dictionary/symptom	Lista symptomów (opcjonalnie filtrowana po gatunku)
PUT	/api/Dictionary/symptom/{id}	Aktualizacja symptomu
DELETE	/api/Dictionary/symptom/{id}	Usunięcie symptomu
POST	/api/Dictionary/treatment	Dodanie typu zabiegu
GET	/api/Dictionary/treatment	Lista wszystkich typów zabiegów
PUT	/api/Dictionary/treatment/{id}	Aktualizacja zabiegu
DELETE	/api/Dictionary/treatment/{id}	Usunięcie zabiegu
POST	/api/Dictionary/disease	Dodanie choroby do słownika
GET	/api/Dictionary/disease	Lista chorób (opcjonalnie filtrowana po gatunku)
PUT	/api/Dictionary/disease/{id}	Aktualizacja choroby
DELETE	/api/Dictionary/disease/{id}	Usunięcie choroby

Bezpieczeństwo

Przepływ autoryzacji

1. **Logowanie** → wydanie JWT (czas życia 10 min) + refresh token (32 bajty losowe, haszowane HMAC-SHA256, zapisane w bazie)
2. **Zapytania API** → JWT w nagłówku `Authorization: Bearer <token>`
3. **Wygasnięcie JWT** → klient wywołuje `/api/Auth/refresh` z wygasłym JWT + refresh token z ciasteczka
4. **Odświeżenie** → walidacja wygasłego JWT przez schemat `ExpiredToken`, rotacja refresh tokena, nowy JWT
5. **Wylogowanie** → unieważnienie refresh tokena w bazie, usunięcie ciasteczka, ustawienie statusu pracownika na Offline

Szyfrowanie

Wrażliwe pola (dane klientów, pracowników, zwierząt, notatki medyczne) są szyfrowane w spoczynku przy użyciu **AES-256-GCM**:

- **Klucz**: SHA-256 z skonfigurowanego `Api:Key`
- **Nonce**: 12 losowych bajtów na operację szyfrowania
- **Tag uwierzytelniający**: 16 bajtów (zabezpieczenie przed manipulacją)
- **Format w bazie**: Base64 `[nonce (12B)][szyfrogram][tag (16B)]`
- **Integralność**: `DataIntegrityException` przy próbie odczytu zmodyfikowanych danych

Haszowanie haseł

Parametry Argon2id:

- **Sól**: 16 losowych bajtów
- **Długość hasha**: 32 bajty
- **Porównanie**: stałoczasowe (odporność na timing attacks)

Tryby środowiska

- **Development**: local file storage, Swagger włączony
- **Production**: PostgreSQL, Azure Blob Storage, telemetria Azure Monitor

Uruchomienie

Wymagania

- .NET 10.0 SDK
- PostgreSQL

Uruchomienie

```
# Przywrócenie zależności
dotnet restore

# Uruchomienie migracji bazy danych
dotnet ef database update

# Start serwera (tryb deweloperski)
dotnet run

# API dostępne pod adresami:
#   https://localhost:65302
#   http://localhost:65303
```

Narzędzia

Projekt używa `dotnet-ef` do migracji (skonfigurowane w `.config/dotnet-tools.json`):

```
# Utworzenie migracji
dotnet ef migrations add <NazwaMigracji>

# Generowanie skryptu SQL z migracji
dotnet ef migrations script
```